```
################################################################################
##################################################     Source Code      #########
################################################################################

import streamlit as st
import gspread
from google.oauth2.service_account import Credentials
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from fpdf import FPDF
import io

# ML imports -- scikit-learn is listed in requirements.txt so Streamlit Cloud ins
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score, classification_report
from sklearn.preprocessing import StandardScaler
import numpy as np

# Define scopes
# The 'scopes' variable defines the level of access the application has to Google
# These specific scopes allow the app to read and write data from Google Sheets a
# enabling secure, real-time interaction with HR-related datasets.
scopes = ["https://www.googleapis.com/auth/spreadsheets", "https://www.googleapis

# Authenticate using Streamlit secrets
# This method uses a JSON configuration stored in Streamlit secrets for authentic
# It's a secure way to handle credentials, ensuring sensitive data is not hard-co
credentials = Credentials.from_service_account_info(
    st.secrets["google_credentials"],
    scopes=scopes
)
client = gspread.authorize(credentials)

# Access Google Sheet
# The application connects to a specific Google Sheet ('Dataset') containing HR d
# This sheet acts as a central storage point for the data, ensuring accessibility
sheet = client.open("Dataset").sheet1
data = sheet.get_all_records()
df = pd.DataFrame(data)  # Converts the data into a Pandas DataFrame for easier m

# Initialize session state for page tracking
```

```
# Streamlit's session state is used to handle page navigation, ensuring a smooth
if "page" not in st.session_state:
    st.session_state.page = "Home"


########################################## Home Page ############################

if st.session_state.page == "Home":

    st.title("⚽ Football Club Performance Monitor")

    st.write(
        "This application provides a comprehensive overview of player performance
        "training engagement, and team composition across all divisions of the cl
    )

    st.write("### Navigate through the application:")

    st.write(
        "- **Dashboard:** Explore interactive visualizations of player data, incl
        "attendance trends, positional analysis, and injury distribution."
    )

    st.write(
        "- **Machine Learning:** Use a predictive model to estimate player perfor
        "such as training attendance, fitness, and goals scored."
    )

    st.write(
        "- **Data Management:** Manage player records efficiently by adding, upda
        "across all divisions."
    )

    st.write(
        "- **Player Report:** Generate a detailed report for individual players,
        "fitness, and development insights."
    )

    st.write("---")

    st.write(
        "This tool is designed to support coaches and club managers in making inf
        "to optimize player development and team performance."
    )

    st.write("### Navigate to:")
    col1, col2, col3, col4 = st.columns([1, 1, 1, 1])
```

```
    with col1:
        if st.button("Dashboard"):
            st.session_state.page = "Dashboard"
    with col2:
        if st.button("Machine Learning"):
            st.session_state.page = "Machine Learning"
    with col3:
        if st.button("Data Management"):
            st.session_state.page = "Data Management"
    with col4:
        if st.button("Division Report"):
            st.session_state.page = "Employee Report"


########################################## Dashboard Page ######################

elif st.session_state.page == "Dashboard":

    st.title("⚽ Player Performance Dashboard")

    if st.button("Homepage"):
        st.session_state.page = "Home"

    # --------------------------------
    # Key Metrics
    # --------------------------------
    st.subheader("Club Overview")

    col1, col2, col3 = st.columns(3)
    col1.metric("Total Players", len(df))
    col2.metric("Avg Performance", round(df["PerformanceScore"].mean(), 1))
    col3.metric("Avg Attendance", f"{round(df['TrainingAttendanceRate'].mean(),1)

    # --------------------------------
    # Age Distribution
    # --------------------------------
    st.subheader("Age Distribution")
    fig, ax = plt.subplots()
    sns.histplot(df['Age'], bins=20, kde=True, ax=ax, color='skyblue')
    ax.set_title("Age Distribution")
    st.pyplot(fig)

    # --------------------------------
    # Position Distribution
    # --------------------------------
    st.subheader("Player Distribution by Position")
    position_counts = df['Position'].value_counts()
```

```
fig, ax = plt.subplots()
position_counts.plot(kind='bar', ax=ax, color='lightblue')
ax.set_title("Players by Position")
ax.set_xlabel("Position")
ax.set_ylabel("Count")
st.pyplot(fig)

# --------------------------------
# Goals by Position
# --------------------------------
st.subheader("Goals by Position")
fig, ax = plt.subplots()
sns.boxplot(x='Position', y='Goals', data=df, ax=ax)
ax.set_title("Goals Distribution by Position")
st.pyplot(fig)

# --------------------------------
# Performance vs Attendance
# --------------------------------
st.subheader("Performance vs Training Attendance")
fig, ax = plt.subplots()
sns.scatterplot(x='TrainingAttendanceRate', y='PerformanceScore', data=df, ax
sns.regplot(x='TrainingAttendanceRate', y='PerformanceScore', data=df, ax=ax,
ax.set_title("Performance vs Attendance")
st.pyplot(fig)

# --------------------------------
# Fitness vs Age
# --------------------------------
st.subheader("Fitness vs Age")
fig, ax = plt.subplots()
sns.scatterplot(x='Age', y='FitnessScore', data=df, ax=ax)
sns.regplot(x='Age', y='FitnessScore', data=df, ax=ax, scatter=False, color='
ax.set_title("Fitness vs Age")
st.pyplot(fig)

# --------------------------------
# Injury Status Distribution
# --------------------------------
st.subheader("Injury Status Distribution")
injury_counts = df['InjuryStatus'].value_counts()
fig, ax = plt.subplots()
injury_counts.plot(kind='bar', ax=ax, color='salmon')
ax.set_title("Injury Status")
st.pyplot(fig)

# --------------------------------
```

```
    # Violin Plot Age Distribution by Gender
    # ------------------------------
    st.subheader("Age Distribution (Male vs Female)")

    fig, ax = plt.subplots()
    sns.violinplot(
        x=["All"] * len(df),
        y="Age",
        hue="Gender",
        data=df,
        split=True,
        inner="quartile",
        ax=ax
    )
    ax.set_xlabel("")
    ax.set_title("Age Distribution Split by Gender")
    st.pyplot(fig)


########################################## Machine Learning Page ###############

elif st.session_state.page == "Machine Learning":

    st.title("⚽ Player Performance Predictor")
    st.write(
        "This page uses **Machine Learning** to predict a player's Performance Sc
        "based on 7 key indicators. The model learns patterns from existing playe
        "and applies them to new inputs."
    )

    if st.button("Homepage"):
        st.session_state.page = "Home"

    # ---------------------------------------------------------------------
    # STEP 1 -- Encode categorical variables
    # InjuryStatus, Position and Gender are text columns (categorical).
    # ML models only understand numbers, so we convert them using Label Encoding:
    #   - Each unique category gets mapped to an integer (e.g. "Fit"→0, "Injured"
    # We work on a copy to avoid modifying the original dataframe.
    # ---------------------------------------------------------------------
    from sklearn.preprocessing import LabelEncoder

    CATEGORICAL = ["InjuryStatus", "Position", "Gender"]
    NUMERICAL   = ["Age", "TrainingAttendanceRate", "FitnessScore", "Goals"]
    FEATURES    = NUMERICAL + CATEGORICAL   # 7 features total
    TARGET      = "PerformanceScore"
```

```
ml_df = df[FEATURES + [TARGET]].dropna().copy()

# Store encoders so we can reuse them on user input later
encoders = {}
for col in CATEGORICAL:
    le = LabelEncoder()
    ml_df[col] = le.fit_transform(ml_df[col].astype(str))
    encoders[col] = le

# Safety check
if len(ml_df) < 10:
    st.warning("Not enough data to train the model (minimum 10 player rows re
    st.stop()

X = ml_df[FEATURES]
y = ml_df[TARGET]

# ---------------------------------------------------------------------
# STEP 2 -- Train / Test Split (80% train, 20% test)
# ---------------------------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ---------------------------------------------------------------------
# STEP 3 -- Feature Scaling
# All 7 features are now numeric. StandardScaler normalises them so
# KNN distance calculations are not dominated by large-range columns
# (e.g. TrainingAttendanceRate 0-100 vs Goals 0-30).
# ---------------------------------------------------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# ---------------------------------------------------------------------
# STEP 4 -- Train KNN Regressor
# ---------------------------------------------------------------------
model = KNeighborsRegressor(n_neighbors=5)
model.fit(X_train_scaled, y_train)

# ---------------------------------------------------------------------
# STEP 5 -- Evaluate
# ---------------------------------------------------------------------
y_pred = model.predict(X_test_scaled)
r2  = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
```

```
# ------------------------------------------------------------------------
# DISPLAY A -- Model accuracy metrics
# ------------------------------------------------------------------------
st.subheader("📊 Model Accuracy")
st.write(
    "The model was trained on **80%** of the player data and tested on the re
    "These scores reflect how well it generalises to new, unseen players."
)
col1, col2, col3 = st.columns(3)
col1.metric("Training rows",  len(X_train))
col2.metric("R² Score",       f"{r2:.2f}")
col3.metric("Avg Error (MAE)", f"{mae:.1f} pts")

# ------------------------------------------------------------------------
# DISPLAY B -- Actual vs Predicted scatter plot
# ------------------------------------------------------------------------
st.subheader("📈 Actual vs. Predicted Performance Score")
st.write("Each dot is a player from the test set. The red line shows a perfec
fig, ax = plt.subplots()
ax.scatter(y_test, y_pred, alpha=0.7, color="steelblue", edgecolors="white",
min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())
ax.plot([min_val, max_val], [min_val, max_val], "r--", label="Perfect predict
ax.set_xlabel("Actual Performance Score")
ax.set_ylabel("Predicted Performance Score")
ax.set_title(f"KNN Predictor  |  R²={r2:.2f}  |  MAE={mae:.1f}")
ax.legend()
st.pyplot(fig)

# ------------------------------------------------------------------------
# DISPLAY C -- Interactive Predictor (7 inputs)
# Numerical features → sliders (same as before)
# Categorical features → selectboxes (new) using the original labels
# ------------------------------------------------------------------------
st.subheader("🎯 Try It: Predict a Player's Score")
st.write("Fill in the player profile below. The model will instantly predict

col1, col2 = st.columns(2)
with col1:
    input_age = st.slider(
        "Age",
        min_value=int(df["Age"].min()),
        max_value=int(df["Age"].max()),
        value=int(df["Age"].median()),
    )
    input_attendance = st.slider(
        "Training Attendance Rate (%)",
```

```
        min_value=float(df["TrainingAttendanceRate"].min()),
        max_value=float(df["TrainingAttendanceRate"].max()),
        value=float(df["TrainingAttendanceRate"].median()),
        step=0.5,
    )
    input_fitness = st.slider(
        "Fitness Score",
        min_value=float(df["FitnessScore"].min()),
        max_value=float(df["FitnessScore"].max()),
        value=float(df["FitnessScore"].median()),
        step=0.5,
    )
    input_goals = st.slider(
        "Goals",
        min_value=int(df["Goals"].min()),
        max_value=int(df["Goals"].max()),
        value=int(df["Goals"].median()),
    )
with col2:
    # Categorical inputs -- show original labels to the user,
    # then encode them exactly as the model was trained.
    input_injury_label   = st.selectbox("Injury Status",
                                        encoders["InjuryStatus"].classes_.tol
    input_position_label = st.selectbox("Position",
                                        encoders["Position"].classes_.tolist(
    input_gender_label   = st.selectbox("Gender",
                                        encoders["Gender"].classes_.tolist())

# Encode user-selected categories using the same LabelEncoders
input_injury   = encoders["InjuryStatus"].transform([input_injury_label])[0]
input_position = encoders["Position"].transform([input_position_label])[0]
input_gender   = encoders["Gender"].transform([input_gender_label])[0]

# Build the 7-feature input row in the correct order
input_array  = np.array([[input_age, input_attendance, input_fitness, input_g
                         input_injury, input_position, input_gender]])
input_scaled = scaler.transform(input_array)
prediction   = model.predict(input_scaled)[0]

st.success(f"**Predicted Performance Score: {prediction:.1f} / 100**")

if prediction >= 75:
    st.info("🌟 Outstanding level -- top performer profile.")
elif prediction >= 50:
    st.info("👍 Good level -- solid player with development potential.")
else:
    st.info("⚠️ Below average -- may benefit from increased training or fitne
```

```
# ======================================================================
# DECISION TREE SECTION -- Classification (High / Medium / Low)
# ======================================================================
st.markdown("---")
st.subheader("🌳 Player Performance Classification")
st.write(
    "Beyond the numeric score, this Decision Tree classifies each player into
    "performance category -- **High**, **Medium**, or **Low** -- based on all
    "The tree learns which thresholds matter most from the data."
)

def categorise(score):
    if score >= 75:   return "High"
    elif score >= 50: return "Medium"
    else:             return "Low"

ml_df["PerformanceCategory"] = ml_df[TARGET].apply(categorise)

X_dt = ml_df[FEATURES]
y_dt = ml_df["PerformanceCategory"]

X_dt_train, X_dt_test, y_dt_train, y_dt_test = train_test_split(
    X_dt, y_dt, test_size=0.2, random_state=42, stratify=y_dt
)

dt_model = DecisionTreeClassifier(max_depth=4, random_state=42)
dt_model.fit(X_dt_train, y_dt_train)

dt_accuracy = dt_model.score(X_dt_test, y_dt_test)

col1, col2 = st.columns(2)
col1.metric("Decision Tree Accuracy", f"{dt_accuracy * 100:.1f}%")
col2.metric("Classes", "High / Medium / Low")

# Tree visualisation
st.write("#### Decision Tree Structure")
st.write(
    "Each node shows the question the model asks. Follow the branches to see
    "how a player ends up classified."
)
fig, ax = plt.subplots(figsize=(22, 8))
plot_tree(
    dt_model,
    feature_names=FEATURES,
    class_names=dt_model.classes_,
    filled=True,
```

```
    rounded=True,
    fontsize=9,
    ax=ax
)
ax.set_title("Decision Tree -- Player Performance Classification (7 Features)
st.pyplot(fig)

# Feature importance
st.write("#### Feature Importance")
st.write("Which of the 7 indicators drive the classification the most?")
importance_df = pd.DataFrame({
    "Feature":    FEATURES,
    "Importance": dt_model.feature_importances_
}).sort_values("Importance", ascending=False)

fig, ax = plt.subplots()
sns.barplot(x="Importance", y="Feature", data=importance_df, ax=ax, palette="
ax.set_title("Feature Importance -- Decision Tree (7 Features)")
ax.set_xlabel("Importance Score")
st.pyplot(fig)

# Classify the player from the inputs above
st.write("#### 🏷️ Classification of Your Player")
st.write("Based on the values you set above, the Decision Tree classifies you

dt_input      = np.array([[input_age, input_attendance, input_fitness, input_
                          input_injury, input_position, input_gender]])
dt_prediction = dt_model.predict(dt_input)[0]
dt_proba      = dt_model.predict_proba(dt_input)[0]
dt_classes    = dt_model.classes_

colour_map = {"High": "🟢", "Medium": "🟡", "Low": "🔴"}
st.success(f"{colour_map.get(dt_prediction, '⚪')} **{dt_prediction} Performe

st.write("Confidence breakdown:")
proba_df = pd.DataFrame({
    "Category":    dt_classes,
    "Probability": (dt_proba * 100).round(1)
}).sort_values("Probability", ascending=False)

fig, ax = plt.subplots(figsize=(5, 2.5))
colours     = {"High": "green", "Medium": "gold", "Low": "salmon"}
bar_colours = [colours.get(c, "steelblue") for c in proba_df["Category"]]
ax.barh(proba_df["Category"], proba_df["Probability"], color=bar_colours)
ax.set_xlabel("Probability (%)")
ax.set_xlim(0, 100)
ax.set_title("Classification Confidence")
```

```
    st.pyplot(fig)

elif st.session_state.page == "Data Management":
    st.title("Data Input Manager (page under construction)")


########################################## Player Report Page ##################

# ──────────────────────────────────────────────────────────────────────
# Player Report Page
# Generates a full individual player card with:
#   - All stats displayed visually
#   - Comparison to team averages
#   - ML performance classification (reuses same model logic)
#   - Downloadable PDF report
# ──────────────────────────────────────────────────────────────────────

elif st.session_state.page == "Employee Report":

    from sklearn.preprocessing import LabelEncoder
    from sklearn.neighbors import KNeighborsRegressor
    from sklearn.tree import DecisionTreeClassifier
    from sklearn.model_selection import train_test_split
    from sklearn.preprocessing import StandardScaler
    from sklearn.metrics import r2_score, mean_absolute_error
    from fpdf import FPDF
    import io
    import tempfile
    import os

    st.title("📋 Player Report")
    st.write("Select a player to generate their individual performance report.")

    if st.button("Homepage"):
        st.session_state.page = "Home"

    # ── Rebuild ML model (same logic as ML page) ──────────────────
    CATEGORICAL = ["InjuryStatus", "Position", "Gender"]
    NUMERICAL   = ["Age", "TrainingAttendanceRate", "FitnessScore", "Goals"]
    FEATURES    = NUMERICAL + CATEGORICAL
    TARGET      = "PerformanceScore"

    ml_df = df[FEATURES + [TARGET]].dropna().copy()
    encoders = {}
    for col in CATEGORICAL:
        le = LabelEncoder()
        ml_df[col] = le.fit_transform(ml_df[col].astype(str))
```

```
    encoders[col] = le

X = ml_df[FEATURES]
y = ml_df[TARGET]

if len(ml_df) >= 10:
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    knn = KNeighborsRegressor(n_neighbors=5)
    knn.fit(X_train_scaled, y_train)

    ml_df_cat = ml_df.copy()
    def categorise(s):
        return "High" if s >= 75 else ("Medium" if s >= 50 else "Low")
    ml_df_cat["PerformanceCategory"] = ml_df_cat[TARGET].apply(categorise)
    X_dt_train, X_dt_test, y_dt_train, y_dt_test = train_test_split(
        ml_df_cat[FEATURES], ml_df_cat["PerformanceCategory"],
        test_size=0.2, random_state=42, stratify=ml_df_cat["PerformanceCatego
    )
    dt_model = DecisionTreeClassifier(max_depth=4, random_state=42)
    dt_model.fit(X_dt_train, y_dt_train)
    ml_ready = True
else:
    ml_ready = False

# ── Player selector ──────────────────────────────────────────
# Build a display label: row index + position + age so coach can identify pla
# (dataset has no player name column -- using index as unique ID)
if "PlayerName" in df.columns:
    player_labels = df["PlayerName"].astype(str).tolist()
    id_col = "PlayerName"
else:
    player_labels = [f"Player {i+1}  |  {df.iloc[i].get('Position','?')}  |
    id_col = None

selected_label = st.selectbox("Choose a player", player_labels)
selected_idx   = player_labels.index(selected_label)
player         = df.iloc[selected_idx]

st.markdown("---")

# ── SECTION 1 : Key Stats ────────────────────────────────────
st.subheader(f"📊 Stats Overview -- {selected_label}")

c1, c2, c3, c4 = st.columns(4)
c1.metric("Performance Score", f"{player.get('PerformanceScore', 'N/A')}")
```

```
c2.metric("Fitness Score",     f"{player.get('FitnessScore', 'N/A')}")
c3.metric("Training Attendance", f"{player.get('TrainingAttendanceRate', 'N/A
c4.metric("Goals", f"{player.get('Goals', 'N/A')}")

c5, c6, c7, c8 = st.columns(4)
c5.metric("Age",            f"{player.get('Age', 'N/A')}")
c6.metric("Position",       f"{player.get('Position', 'N/A')}")
c7.metric("Injury Status",  f"{player.get('InjuryStatus', 'N/A')}")
c8.metric("Gender",         f"{player.get('Gender', 'N/A')}")

# ── SECTION 2 : Comparison radar-style bar chart ──────────────
st.subheader("📈 Comparison vs. Team Average")

num_cols = ["PerformanceScore", "FitnessScore", "TrainingAttendanceRate", "Go
available = [c for c in num_cols if c in df.columns]

player_vals = [float(player.get(c, 0)) for c in available]
team_avg    = [float(df[c].mean()) for c in available]

x     = np.arange(len(available))
width = 0.35

fig, ax = plt.subplots(figsize=(9, 4))
bars1 = ax.bar(x - width/2, player_vals, width, label="This Player", color="s
bars2 = ax.bar(x + width/2, team_avg,    width, label="Team Average", color="
ax.set_xticks(x)
ax.set_xticklabels(available, rotation=15)
ax.set_title("Player vs. Team Average")
ax.legend()
ax.bar_label(bars1, fmt="%.1f", padding=2, fontsize=8)
ax.bar_label(bars2, fmt="%.1f", padding=2, fontsize=8)
st.pyplot(fig)

# ── SECTION 3 : Where does this player rank? ──────────────────
st.subheader("🏆 Team Ranking")

rank_col = "PerformanceScore"
if rank_col in df.columns:
    df_ranked   = df.copy().reset_index(drop=True)
    df_ranked["_rank"] = df_ranked[rank_col].rank(ascending=False, method="mi
    player_rank = int(df_ranked.iloc[selected_idx]["_rank"])
    total       = len(df_ranked)
    percentile  = round((1 - (player_rank - 1) / total) * 100, 1)

    rc1, rc2, rc3 = st.columns(3)
    rc1.metric("Rank in Club",   f"#{player_rank} / {total}")
    rc2.metric("Top percentile", f"{percentile}%")
```

```
    rc3.metric("Score vs Best",
               f"{float(player.get(rank_col,0)) - float(df[rank_col].max()):.

# ── SECTION 4 : ML Prediction ──────────────────────────────────
st.subheader("🤖 ML Performance Prediction")

if ml_ready:
    try:
        row_encoded = []
        for col in FEATURES:
            val = player.get(col, None)
            if col in CATEGORICAL:
                val = encoders[col].transform([str(val)])[0]
            row_encoded.append(float(val))

        row_array  = np.array([row_encoded])
        row_scaled = scaler.transform(row_array)

        predicted_score    = knn.predict(row_scaled)[0]
        predicted_category = dt_model.predict(row_array)[0]
        proba              = dt_model.predict_proba(row_array)[0]
        classes            = dt_model.classes_

        colour_map = {"High": "🟢", "Medium": "🟡", "Low": "🔴"}
        pc1, pc2, pc3 = st.columns(3)
        pc1.metric("Predicted Score",    f"{predicted_score:.1f} / 100")
        pc2.metric("Actual Score",       f"{float(player.get(TARGET, 0)):.1f}
        pc3.metric("ML Category", f"{colour_map.get(predicted_category,'')} {

        # Confidence bar
        proba_df = pd.DataFrame({"Category": classes, "Probability": (proba*1
        proba_df = proba_df.sort_values("Probability", ascending=False)
        fig2, ax2 = plt.subplots(figsize=(5, 2))
        clr = {"High":"green","Medium":"gold","Low":"salmon"}
        ax2.barh(proba_df["Category"], proba_df["Probability"],
                 color=[clr.get(c,"steelblue") for c in proba_df["Category"]]
        ax2.set_xlim(0, 100)
        ax2.set_xlabel("Probability (%)")
        ax2.set_title("Classification Confidence")
        st.pyplot(fig2)

    except Exception as e:
        st.warning(f"Could not run ML prediction for this player: {e}")
else:
    st.info("Not enough data to run ML prediction (need at least 10 players).

# ── SECTION 5 : Injury history visual ──────────────────────────
```

```
if "InjuryStatus" in df.columns:
    st.subheader("🏥 Injury Status vs. Position Group")
    fig3, ax3 = plt.subplots(figsize=(7, 3))
    inj_counts = df.groupby(["Position","InjuryStatus"]).size().unstack(fill_
    inj_counts.plot(kind="bar", ax=ax3, color=["salmon","lightgreen"])
    ax3.set_title("Injury Status by Position")
    ax3.set_xlabel("Position")
    ax3.tick_params(axis="x", rotation=15)

    # Highlight the current player's position
    positions = list(inj_counts.index)
    player_pos = str(player.get("Position",""))
    if player_pos in positions:
        bar_idx = positions.index(player_pos)
        ax3.axvline(x=bar_idx, color="steelblue", linewidth=2.5,
                    linestyle="--", label=f"← {player_pos} (this player)")
        ax3.legend()
    st.pyplot(fig3)

# ── SECTION 6 : PDF Export ──────────────────────────────────
st.markdown("---")
st.subheader("📄 Download Report as PDF")

if st.button("Generate PDF Report"):
    try:
        pdf = FPDF()
        pdf.add_page()

        # Header
        pdf.set_fill_color(20, 83, 45)   # dark green
        pdf.set_text_color(255, 255, 255)
        pdf.set_font("Arial", "B", 18)
        pdf.cell(0, 14, "Football Club -- Player Report", ln=True, fill=True,
        pdf.ln(4)

        # Player identity
        pdf.set_text_color(0, 0, 0)
        pdf.set_font("Arial", "B", 13)
        pdf.cell(0, 9, f"Player: {selected_label}", ln=True)
        pdf.set_font("Arial", "", 11)
        pdf.cell(0, 7, f"Position: {player.get('Position','N/A')}    |    Age
        pdf.ln(4)

        # Stats table
        pdf.set_font("Arial", "B", 12)
        pdf.set_fill_color(220, 240, 220)
        pdf.cell(0, 8, "Performance Metrics", ln=True, fill=True)
```

```
pdf.set_font("Arial", "", 11)

stats = {
    "Performance Score":     player.get("PerformanceScore", "N/A"),
    "Fitness Score":         player.get("FitnessScore", "N/A"),
    "Training Attendance":   f"{player.get('TrainingAttendanceRate','
    "Goals":                 player.get("Goals", "N/A"),
}
for label, val in stats.items():
    pdf.cell(80, 7, label, border=1)
    pdf.cell(0,  7, str(val), border=1, ln=True)
pdf.ln(4)

# Team ranking
if rank_col in df.columns:
    pdf.set_font("Arial", "B", 12)
    pdf.set_fill_color(220, 240, 220)
    pdf.cell(0, 8, "Team Ranking", ln=True, fill=True)
    pdf.set_font("Arial", "", 11)
    pdf.cell(0, 7, f"Rank: #{player_rank} out of {total} players  |
    pdf.ln(4)

# ML section
if ml_ready:
    pdf.set_font("Arial", "B", 12)
    pdf.set_fill_color(220, 240, 220)
    pdf.cell(0, 8, "ML Performance Prediction", ln=True, fill=True)
    pdf.set_font("Arial", "", 11)
    try:
        pdf.cell(0, 7, f"Predicted Score: {predicted_score:.1f} / 100
        pdf.cell(0, 7, f"Performance Category: {predicted_category}",
        pdf.cell(0, 7, f"Actual Score: {float(player.get(TARGET, 0)):
    except:
        pdf.cell(0, 7, "ML prediction not available for this player."
    pdf.ln(4)

# Comparison chart -- save to temp file and embed
fig_pdf, ax_pdf = plt.subplots(figsize=(7, 3))
ax_pdf.bar(np.arange(len(available)) - 0.175, player_vals, 0.35,
           label="This Player", color="steelblue")
ax_pdf.bar(np.arange(len(available)) + 0.175, team_avg, 0.35,
           label="Team Average", color="lightgray")
ax_pdf.set_xticks(np.arange(len(available)))
ax_pdf.set_xticklabels(available, rotation=15)
ax_pdf.legend()
ax_pdf.set_title("Player vs. Team Average")
```

```
        with tempfile.NamedTemporaryFile(suffix=".png", delete=False) as tmp:
            tmp_path = tmp.name
        fig_pdf.savefig(tmp_path, bbox_inches="tight", dpi=120)
        plt.close(fig_pdf)

        pdf.set_font("Arial", "B", 12)
        pdf.set_fill_color(220, 240, 220)
        pdf.cell(0, 8, "Comparison Chart", ln=True, fill=True)
        pdf.image(tmp_path, x=15, w=170)
        os.unlink(tmp_path)

        # Footer
        pdf.set_y(-15)
        pdf.set_font("Arial", "I", 8)
        pdf.set_text_color(120, 120, 120)
        pdf.cell(0, 10, "Generated by Football Club Performance Monitor", ali

        pdf_bytes = pdf.output(dest="S").encode("latin-1")
        st.download_button(
            label="📩 Download PDF",
            data=pdf_bytes,
            file_name=f"player_report_{selected_label.replace(' ','_')}.pdf",
            mime="application/pdf"
        )

    except Exception as e:
        st.error(f"PDF generation failed: {e}")
```